WEEK 05 · BLOCK 5A

Block 5a — Values & Functions

Goal: prove JavaScript's quirks to yourself in code — the `==` trap, truthy/falsy values and optionals, and functions as values you can store, pass, and return.

Time: ~45 minutes · **You need:** a terminal with Node (from Block 0)

Setup

This is a plain Node script — no install needed.

```
cd exercises/week05/5a/starter
node script.js        # or: npm start
```

Edit `script.js`, save, and re-run to see your changes. Everything prints to the terminal.

> No browser today. `console.log` is your microscope — log early, log often.

Task 1 — Equality

- **1a:** For each comparison, **write your prediction** in the comment *before* running. Then run and check. Any surprises are exactly the point.
- **1b:** `isAgeMissing` uses `==` and wrongly treats `"0"` as missing. **Fix it** so `isAgeMissing("0")` is `false` but `isAgeMissing("")` is `true`. (Hint: `===`, and think about what "missing" really means.)

Task 2 — Truthiness & optionals

- **2a:** Loop over `values` and print each one as **truthy** or **falsy**. Predict the six falsy ones first.
- **2b:** Reading `user.address.city` would crash (`user` has no `address`). Use **optional chaining** (`?.`) so it safely yields `undefined`.

Task 3 — Functions as values

- **3a:** Store a `square` function in a variable.
- **3b:** Write `applyTwice(fn, x)` → `fn(fn(x))`. Check `applyTwice(square, 3)` is `81` (a function **passed as an argument**).
- **3c:** Write `adder(n)` that **returns a function** adding `n`. Check `adder(10)(5)` is `15` (a function **returned** from a function).

Requirements checklist

- [ ] Predicted all of Task 1a, then verified against the output
- [ ] `isAgeMissing("0")` is `false`, `isAgeMissing("")` is `true`
- [ ] Each value in Task 2a labelled truthy/falsy correctly
- [ ] `city` is `undefined` (no crash)
- [ ] `applyTwice(square, 3)` → `81`; `adder(10)(5)` → `15`

Stretch goals (optional)

- Rewrite `applyTwice` as an **arrow function**.
- Write `compose(f, g)` returning a function that computes `f(g(x))`.
- Explore: what does `[] == ![]` evaluate to? Explain *why* using coercion — then be glad you use `===`.